郑州轻工业大学

本科毕业设计（论文）

题　　目　校园物资智能管理系统设计与实现

学生姓名　　刘祎凯

专业班级　　软件工程 22-06 班

学　　号　　542213340621

学　　院　　软件学院

指导教师(职称)　　周开来

完成时间　　20　年　月　　日

郑州轻工业大学

本科毕业设计（论文）任务书

题目　　校园物资智能管理系统设计与实现

专业　　软件工程　　学号　542213340621 姓名　刘祎凯

主要内容、基本要求、主要参考资料等：

一、 主要内容

本课题围绕校园物资管理信息化需求，设计并实现“校园物资智能管理系统”。系统采用前后端分离架构，后端基于 Spring Boot + MyBatis-Plus + MySQL，前端基于 Vue3 + Vite，主要完成以下内容：

1. 完成系统需求分析、总体架构设计与数据库设计；
2. 实现基础数据管理（用户、角色、部门、校区、仓库、库位、物资分类与档案）；
3. 实现核心业务功能（入库、出库、调拨、申领审批、库存批次与效期管理）；
4. 实现预警与分析功能（低库存、临期预警、统计报表与可视化）；
5. 实现系统安全与审计（基于 Spring Security + JWT 的认证授权、登录日志与操作日志）；
6. 完成系统测试、性能优化与毕业论文撰写。

二、基本要求

1. 系统功能完整，业务流程清晰，满足校园物资管理实际应用场景；
2. 数据库设计规范，关键字段与索引合理，具备基本并发与查询性能；
3. 前后端交互稳定，主要页面与接口运行正常，具备较好可用性；
4. 代码结构清晰、命名规范、注释完整，具备可维护性与可扩展性；
5. 毕业论文内容完整、逻辑严谨、格式规范，符合学校毕业设计规范。

三、思政要求

1. 坚持服务校园治理与师生需求，体现信息化建设的社会责任；
2. 强化数据安全与权限边界意识，落实最小权限和审计追踪机制；

3. 在系统设计中体现协同管理、规范管理和应急保障意识；
4. 培养严谨求实、诚信科研、规范开发的职业素养。

四、主要参考资料

[1] 王伟, 刘明. 基于 Spring Boot 的后端架构设计与性能优化研究[J]. 计算机系统应用, 2021, 30(5): 102-109.

[2] 陈志敏, 何建国. 大数据背景下智慧校园建设与管理模式创新探讨[J]. 现代教育管理, 2020, 12(8): 45-51.

[3] 张强, 李红梅. 高校突发公共事件应急物资储备体系构建研究[J]. 中国安全生产科学技术, 2022, 18(3): 67-73.

[4] 李东, 赵莉. Vue3+Vite 开发实战: 组合式 API 与工程化实践[M]. 北京: 人民邮电出版社, 2023: 155-180.

[5] 刘强, 周波. MyBatis-Plus 企业级高可用开发指南[M]. 电子工业出版社, 2021: 89-105

完 成 期 限：

指导教师签名：

专业负责人签名：

年 月 日

校园物资智能管理系统设计与实现

摘　　要

面对高校物资台账分散、审批链条较长、库存批次追踪困难和风险处置滞后的管理问题，本文围绕校园物资智能管理系统的设计与实现展开研究。系统采用前后端分离架构，前端基于 Vue 3、Pinia、Element Plus 和 ECharts 构建业务页面与统计展示，后端基于 Spring Boot、Spring Security、JWT、MyBatis-Plus 和 JdbcTemplate 实现认证授权、业务流程控制与数据访问，数据层使用 MySQL。系统围绕校园物资管理场景，完成了基础数据维护、库存与批次管理、申领审批、调拨协同、预警处理、通知日志和统计分析等核心功能。本文所述“智能”主要体现在规则化和轻量算法支持下的业务辅助能力，包括预警自动扫描、调拨推荐、补货建议以及按效期优先的批次执行。该系统实现了对校园物资全流程的规范化管理，提升了业务协同效率、库存可视性和风险响应能力，可为高校物资管理的信息化建设提供参考。

关键词：校园物资管理；库存批次；调拨推荐；库存预警；补货建议

Design and Implementation of Campus Material Intelligent Management System

ABSTRACT

This thesis focuses on the design and implementation of a campus material intelligent management system for problems such as scattered ledgers, lengthy approval processes, weak batch traceability, and delayed risk handling in campus material management. The system adopts a front-end/back-end separation architecture. The front end is built with Vue 3, Pinia, Element Plus, and ECharts, while the back end is built with Spring Boot, Spring Security, JWT, MyBatis-Plus, and JdbcTemplate, with MySQL as the persistence layer. The system implements core functions including master data management, inventory and batch management, requisition approval, transfer coordination, warning handling, notification and log management, and statistical analysis. In this thesis, the intelligent aspect of the system is reflected in rule-based and lightweight algorithm-assisted capabilities, including automatic warning scanning, transfer recommendation, replenishment suggestion, and expiry-oriented batch execution. The system provides a complete digital workflow for campus material management and improves business coordination, inventory visibility, and risk response efficiency, which can serve as a practical reference for the informatization of campus material management.

Keywords: campus material management; inventory batch; transfer recommendation; inventory warning; replenishment suggestion

目 录

摘 要 I
ABSTRACT II
1 绪论 1
1.1 研究背景 1
1.2 研究意义 1
1.3 国内外研究现状 2
1.3.1 国内研究现状 2
1.3.2 国外相关实践启示 2
1.4 研究内容与论文结构 3
2 相关技术介绍 4
2.1 技术选型概述 4
2.2 前端开发技术 4
2.3 后端与数据访问技术 4
3 系统需求分析 5
3.1 业务场景与角色划分 5
3.2 功能需求分析 5
3.2.1 基础数据管理需求 5
3.2.2 库存与批次管理需求 5
3.2.3 申领审批与出库需求 6
3.2.4 调拨协同需求 6
3.2.5 预警、统计与支撑功能需求 6
3.3 非功能需求分析 7

3.4 业务流程与用例分析 7
3.4.1 申领审批用例分析 7
3.4.2 调拨执行用例分析 9
3.4.3 预警处置与统计分析用例分析 11
4 系统总体设计 14
4.1 系统总体架构 14
4.2 功能模块设计 15
4.3 接口与安全设计 16
4.3.1 接口设计 17
4.3.2 认证与授权设计 17
4.3.3 安全与审计设计 17
4.4 智能能力设计 17
5 数据库设计 19
5.1 数据库设计原则 19
5.2 实体关系设计 19
5.3 关键数据表设计 21
6 系统详细设计与实现 31
6.1 开发环境与运行环境 31
6.2 认证授权与登录态管理实现 32
6.3 基础数据管理实现 34
6.4 库存、批次、出入库实现 35
6.5 申领审批与调拨实现 35
6.5.1 申领审批实现 35

6.5.2 调拨实现与推荐机制 37
6.6 预警、补货建议与统计分析实现 39
6.7 日志、通知与事件管理实现 42
7 系统测试 44
7.1 测试环境与验证方法 44
7.2 自动化测试结果 44
7.3 业务场景验证 45
7.4 构建验证与问题边界 49
结束语 51
致　　谢 52
参考文献 53
附录 1 关键接口与测试补充说明 55

1 绪论

1.1 研究背景

高校物资管理同时面对日常保障和突发事件两类场景：一方面，实验耗材、医疗防护、应急照明、清洁消杀等物资需要围绕部门申领、仓库出入库和批次追踪持续运转；另一方面，物资配置又必须服务于校园安全、公共卫生和大型活动保障等时效性任务。随着智慧校园建设逐步由网络基础设施扩展到业务协同层，传统依赖纸质台账、分散 Excel 表和人工电话确认的管理方式越来越难以支撑跨部门、跨校区的物资流转要求[1][2]。

从本项目代码与 SQL 结构可以看到，系统需要同时管理部门、角色、用户、校区、仓库、库位、分类、物资档案、库存、库存批次、申领单、调拨单、预警记录、通知消息和日志审计等多类对象。这些对象之间并不是孤立存在的：部门用户发起申领后需要经过审批人审核，审核通过后还要由仓库管理员结合库存与批次情况执行出库，最终再由使用方完成签收闭环；在调拨场景中，还存在调出仓与调入仓之间的库存协同、运输路径选择和状态流转问题。

系统已经形成较完整的前后端实现与自动化测试基础。本文将校园物资智能管理系统界定为面向高校日常保障与应急物资协同的 B/S 架构业务系统，其智能化能力主要体现在基于库存、批次、出库历史和校区路径数据的规则分析、辅助决策与趋势预测，整体控制在规则与统计分析范围内。

1.2 研究意义

本课题的研究意义在于将校园物资管理中的申领、审批、出库、签收、调拨、预警和日志留痕等环节统一到可追踪的数据流程中，并通过预警扫描、调拨推荐、补货建议和效期优先批次规则，将自动感知与辅助决策能力嵌入主业务链路。

从应用角度看，项目的价值主要落在角色协作、责任确认和风险前移上。SecurityConfig、AuthService、WarningService 和 TransferService 说明，不同角色的菜单边界、预警处置入口和候选来源仓推荐都已经被明确落实。对校园场景来说，这类可解释的规则型智能比单纯展示几个管理页面更有实际意义。

从工程实践角度看，系统围绕真实业务对象建立数据模型和状态流转规则，使功能设计能够与代码实现、数据库结构和测试验证保持一致。该研究方式有助于提升毕业设计成果的可实现性、可验证性和可维护性。

1.3 国内外研究现状

1.3.1 国内研究现状

国内围绕智慧校园和高校信息化的研究已经积累了大量案例，常见切入点包括统一身份认证、流程电子化、后勤资源协同和数据共享平台建设[1]。在具体实现层面，基于 Spring Boot 与 Vue 的前后端分离方案已经成为高校业务系统中的常见组合，用于支撑角色隔离、列表查询、表单流转和基础统计展示[3][4][5][6]。

和本项目更接近的研究，多分布在图书、采购、资产和后勤物资等资源管理方向[7][8][9]。这些成果通常能够提供比较成熟的主数据维护、库存统计和审批流建模思路，但对批次追踪、临期处置和复杂状态迁移的讨论往往不够细，很多论文停留在功能截图层面。

从这一背景看，本项目的可取之处不在于提出了新的理论模型，而在于把校园物资管理里真正需要落地的对象和流程接上了代码实现。论文在此基础上展开分析，更容易把需求、设计和实现对应起来。

1.3.2 国外相关实践启示

国外公开资料里，与本项目最相关的并不是某一篇现成的校园物资论文，而是框架文档、工程规范和通用的业务系统实践。Spring Boot、Vue、Vite 和 JWT 相关文档提供的是实现路径：如何组织模块、如何处理认证、如何管理依赖、如何保持接口一致性[10][11][12][13]。

这类资料的共同特点是强调主业务链路的稳定性和工程实现的可维护性。本系统在实现上也采取了类似思路，优先完成登录鉴权、申领审批、库存批次、调拨推荐、预警生成和统计展示等核心流程。

在资源调度与路径规划领域，最短路径算法长期被用于候选路线筛选与运输路径决策。Dijkstra 算法因计算过程清晰、实现成本低、结果可解释，适合节点规模有限的校园多校区调拨场景[14]。本项目将其用于候选来源仓排序，不扩展为复杂物流优化系统。

因此，本文对国外相关实践的吸收主要体现在工程方法上，包括模块化组织、接口一致性、前后端分离和认证规范等方面。这种处理方式更符合校园物资管理系统的实际规模，也有利于形成边界明确的本科毕业设计成果。

1.4 研究内容与论文结构

本文围绕校园物资智能管理系统展开。正文先分析角色分工、业务场景和功能边界，再结合数据库结构、前后端依赖和统一响应机制说明总体设计，随后重点阐述认证授权、基础数据、库存批次、申领审批、调拨执行、预警分析和日志通知等核心实现。

结构安排上，第 1 章阐述选题背景、研究意义和相关实践；第 2 章介绍系统采用的相关技术；第 3 章分析系统需求与业务流程；第 4 章说明系统总体架构、功能模块和权限设计；第 5 章阐述数据库设计；第 6 章展开关键功能实现；第 7 章说明测试环境、测试方法与验证结果；最后总结系统完成情况和后续可扩展方向。

全文以系统已实现功能为基础展开论述。未形成实现支撑的性能、部署、推广和运行成效不作为本文结论。

2 相关技术介绍

2.1 技术选型概述

系统前端采用 Vue 3、Pinia、Element Plus、Axios、Vue Router 和 ECharts，使用 Vite 作为构建工具；后端采用 Spring Boot 3.3.5、Spring Security、MyBatis-Plus、JdbcTemplate 和 JWT；数据层使用 MySQL 8，并在测试阶段引入 H2 内存数据库。上述技术组合对应的是当前高校信息化项目中常见的前后端分离路线：前端负责页面交互与图表展示，后端负责认证授权、业务编排和数据持久化[9][10][11][12][13][15][16][17][18][19]。

2.2 前端开发技术

选择 Vue 3 与 Pinia 的原因，在于项目存在较多角色菜单、表单状态和图表页面，需要较清晰的组件组织与轻量状态管理。结合 `package.json` 依赖范围与本次 `npm ls` 结果，可以确认当前前端实际安装的核心版本包括 Vue 3.5.29、Pinia 2.3.1、Vite 6.4.1、Element Plus 2.13.5 和 ECharts 5.6.0，这意味着前端工程已经使用组合式 API 与现代化构建链路。Element Plus 承担表单、表格、对话框、消息提示等通用界面元素，ECharts 提供库存与出入库统计图表的呈现能力[11][12][16][18]。

2.3 后端与数据访问技术

后端选择 Spring Boot 3 的主要原因，是其在 Web API、依赖管理、配置组织和测试支持方面较为成熟。Spring Security 用于认证与权限控制，MyBatis-Plus 负责实体映射和通用 CRUD，JdbcTemplate 用于统计与智能分析中的聚合 SQL。对于本科项目而言，这种组合既能覆盖典型企业级开发流程，又不会引入系统未实际使用的复杂中间件依赖，因此符合“功能完整、实现清晰、可测试”的设计目标[9][10][13][15][19]。

3 系统需求分析

3.1 业务场景与角色划分

从 seed.sql、菜单装配逻辑和前端路由可以看出，系统真正围绕四类角色运转：系统管理员、仓库管理员、审批人和部门用户。管理员负责组织与主数据维护，仓库管理员处理入库、出库、批次和调拨执行，审批人处理审核动作，部门用户则承担申领、查看和签收这一侧的业务操作。

业务场景并不是平均铺开的。系统的主线首先是库存与批次管理，其次是部门申领到仓库执行的闭环流转，再往上才是跨仓调拨、预警生成和统计分析。这种层次关系与仓库表结构、服务拆分和页面入口是一致的，不是论文后加出来的抽象分类。

角色边界不仅体现在业务说明中，也体现在接口权限和菜单装配机制中。SecurityConfig、控制器上的 PreAuthorize 表达式和 AuthService.buildMenusByRole 共同决定用户可访问的菜单、接口和工作流页面，因此需求分析需要将权限约束作为系统运行的基本前提。

3.2 功能需求分析

3.2.1 基础数据管理需求

基础数据并不是陪衬模块。部门、角色、用户、校区、仓库、库位、分类、物资、供应商和系统配置这些对象一旦定义不稳，后续单据和统计都无法成立。前端已经为这些对象提供了独立页面，说明它们在系统里需要被长期维护，而不是一次性导入后就不再变化。

其中影响面最大的是物资档案。material_info 保存的编码、名称、分类、规格、单位、安全库存、保质期和供应商信息，会反复进入库存、预警、申领和补货建议等逻辑，所以这一部分的需求重点其实是主数据一致性，而不是单个表单页面本身。

3.2.2 库存与批次管理需求

校园物资管理不能只看一个总库存数字。inventory 负责汇总某物资在某仓库下的当前量和锁定量，inventory_batch 继续把来源批次、剩余数量、生产日期和到期日期拆开记录。这意味着系统在需求上必须同时支撑总账视图和批次视图，否则临期处理与先到期先出都无从谈起。

对应到操作层面，系统至少要能完成入库建批次、库存查询、按批次排序和出库扣减。InventoryService 里围绕批次排序和数量校验的实现也说明，这里需要的是可执行的业务规则，而不是一句笼统的“支持库存管理”。

3.2.3 申领审批与出库需求

申领审批是系统最核心的入口。apply_order 和 apply_order_item 采用主单加明细的结构，把申请部门、申请人、紧急等级、原因、场景和物资明细放进同一张业务单据里。围绕这张单据形成的状态链条覆盖了 DRAFT、SUBMITTED、APPROVED、REJECTED、OUTBOUND 和 RECEIVED 六个阶段，需求分析必须与这一真实状态模型保持一致。

代码里还能看到另一个校园场景：紧急申领。ApplyService 会根据 urgency_level 触发快速审批分支，所以系统不仅要服务于计划性领用，还要支撑临时保障、夜间处置和突发场景下更短的审批路径，同时留下审批意见和时间记录，避免流程缩短后失去追踪能力。

3.2.4 调拨协同需求

当一端仓库短缺、另一端仍有余量时，系统不能只让管理员手动改两个库存字段。transfer_order 和 transfer_order_item 表明，调拨必须先形成单据，再经历提交、审批、执行和签收，调出仓和调入仓的变化要在同一条业务语义下完成。

与普通出入库不同，调拨还要回答“从哪调、走哪条路径”这样的问题。项目里的 DijkstraUtil 与 /api/transfer/recommend 接口说明，系统会以 targetCampus、materialId 和 qty 作为输入，先计算目标校区到各候选校区的最短距离，再筛选 currentQty >= qty 的库存记录，并按距离升序返回候选来源仓。因此，这里的智能推荐本质上是基于校区图最短路径和库存可用量的候选仓排序，而不是物流调度优化。

3.2.5 预警、统计与支撑功能需求

要让校园物资系统长期可用，只把单据流转做通还不够。warning_record、notification、event_record、login_log 和 operation_log 这些表说明，预警、消息、事件和日志不是附属功能，而是支撑主链路运行的独立模块。库存不足、库存积压、临期、过期和异常领用等预警类型也都需要在需求阶段提前明确。

统计与智能分析需求可进一步拆为三类：第一类是定时预警扫描，围绕低库存、积压、临期、过期和异常领用主动发现风险；第二类是补货建议与消耗预测，围绕近 30 天出库量、近 6 个月历史和 safetyStock 计算目标库存与预测值；第三类是统计聚合展示，围绕库存分布、趋势和排行生成管理视图。它们共同依赖库存表、批次表、出库历史和安全库存阈值，因此前面的主数据、库存和单据记录必须保持完整，否则分析结果没有基础。

3.3 非功能需求分析

非功能需求里最先落地的是安全与权限隔离。后端通过 Spring Security 过滤链、JWT、Refresh Token 和方法级权限控制保护接口访问，前端通过本地 token 存储、路由守卫和 401 处理维持登录态。换句话说，系统首先要保证不同角色只能进入各自职责范围内的页面和操作链路。

另一个现实要求是可维护性。项目采用前后端分离组织方式，后端服务按认证、库存、申领、调拨、预警、智能分析和日志等模块拆分，数据库表统一保留 deleted、version、created_at 和 updated_at 一类通用字段。这种组织方式更接近可持续演进的业务系统，而不是一次性演示项目。

系统还需要具备可测试性。项目提供 H2 测试配置，并围绕认证、申请、调拨、预警与异常处理设置自动化测试类，使核心业务逻辑能够在统一测试环境下重复验证。并发能力和性能指标则需要依赖专门的压测数据进一步评价。

3.4 业务流程与用例分析

3.4.1 申领审批用例分析

在申领审批用例中，主要参与者包括部门用户、审批人和仓库管理员。部门用户首先创建申领单，录入申请原因、使用场景、紧急等级和物资明细；草稿单在提交前可以修改，提交后状态变为 `SUBMITTED`。审批人对单据进行审核，若不符合要求则退回为 `REJECTED` 并记录意见；若审核通过，则状态进入 `APPROVED`，仓库管理员据此执行出库。

出库环节并非单纯的状态修改，而是要结合库存与批次完成实物分配。系统根据库存和批次记录生成出库单与出库明细，回写申领明细的 `actual_qty`，并把申领单状态

更新为 `OUTBOUND`。最终，部门用户对实际领用结果进行签收，状态更新为 `RECEIVED`，一张申领单才算闭环结束。

申领审批的主要参与者、状态推进与出库签收关系可概括为图 3-1 所示。

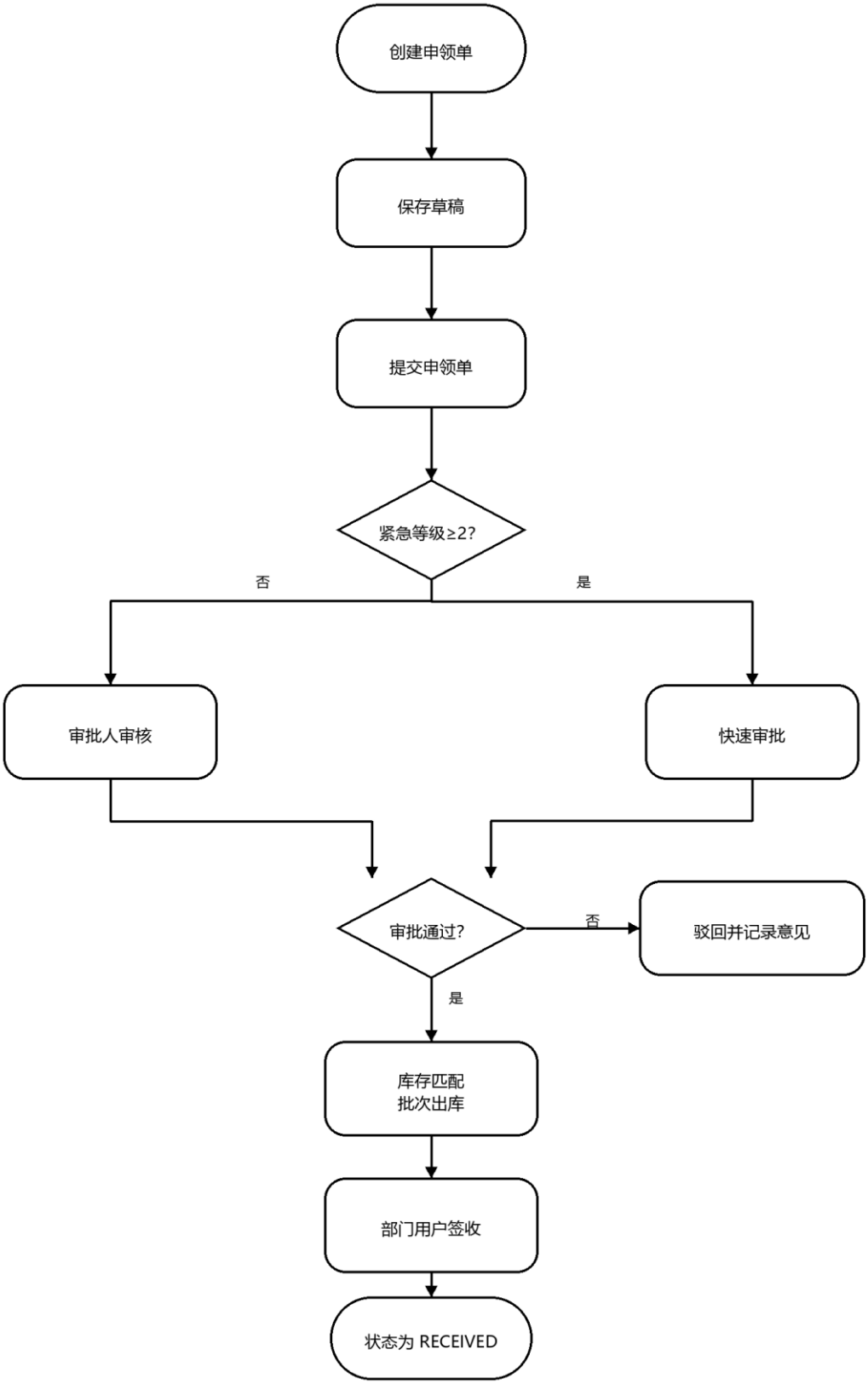

图 3-1 申领审批闭环流程图

3.4.2 调拨执行用例分析

调拨执行用例从仓库管理员发起单据开始。管理员根据目标仓库存量或保障需求创建调拨单，再调用推荐接口获取候选来源仓。推荐阶段以目标校区为源点进行最短路径计算，再叠加库存可用量条件筛选满足 currentQty >= qty 的仓库，最终按距离升序给出候选结果。审批通过后，系统进入执行阶段，同时处理调出仓扣减、调入仓补入和状态推进，而不是只改动一个库存总量字段。

这类业务对一致性的要求比普通申领更高，因为一次调拨会同时影响两端仓库。目标仓完成签收前，系统不会把单据视为最终结束。通过这种设计，仓间转移被组织为申请、审核、执行、签收四段式链路。

调拨执行用例中从推荐候选仓到签收完成的状态推进关系可概括为图 3-2 所示。

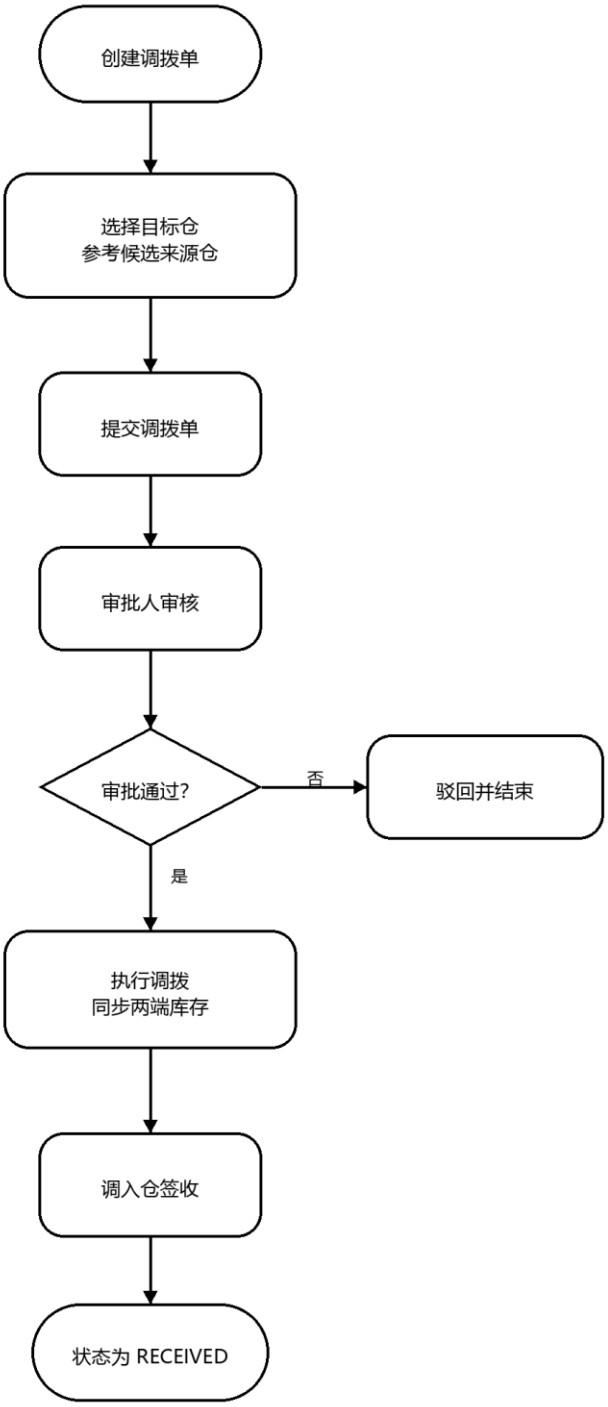

图 3-2 调拨执行流程图

3.4.3 预警处置与统计分析用例分析

预警处置用例由系统主动触发。WarningService.scan() 周期性检查库存、批次和出库历史信息，按 5 类固定规则生成库存不足、库存积压、临期、过期和异常领用记录，并通过未处理状态去重避免重复告警。管理员或仓库管理员在预警页查看待处理项后，可以补录处理说明并更新状态；若需要通知相关角色，系统还会写入消息表供后续查看。

统计分析则服务于管理决策。系统一方面把出入库流水、库存分布和时间维度聚合到图表页与大屏页，另一方面由 SmartService 基于历史出库记录返回补货建议和月度预测结果。它既是一个独立功能，也是前面主数据、库存和单据记录是否规范的反向检验。

预警生成、人工处置与通知联动之间的关系可概括为图 3-3 所示。

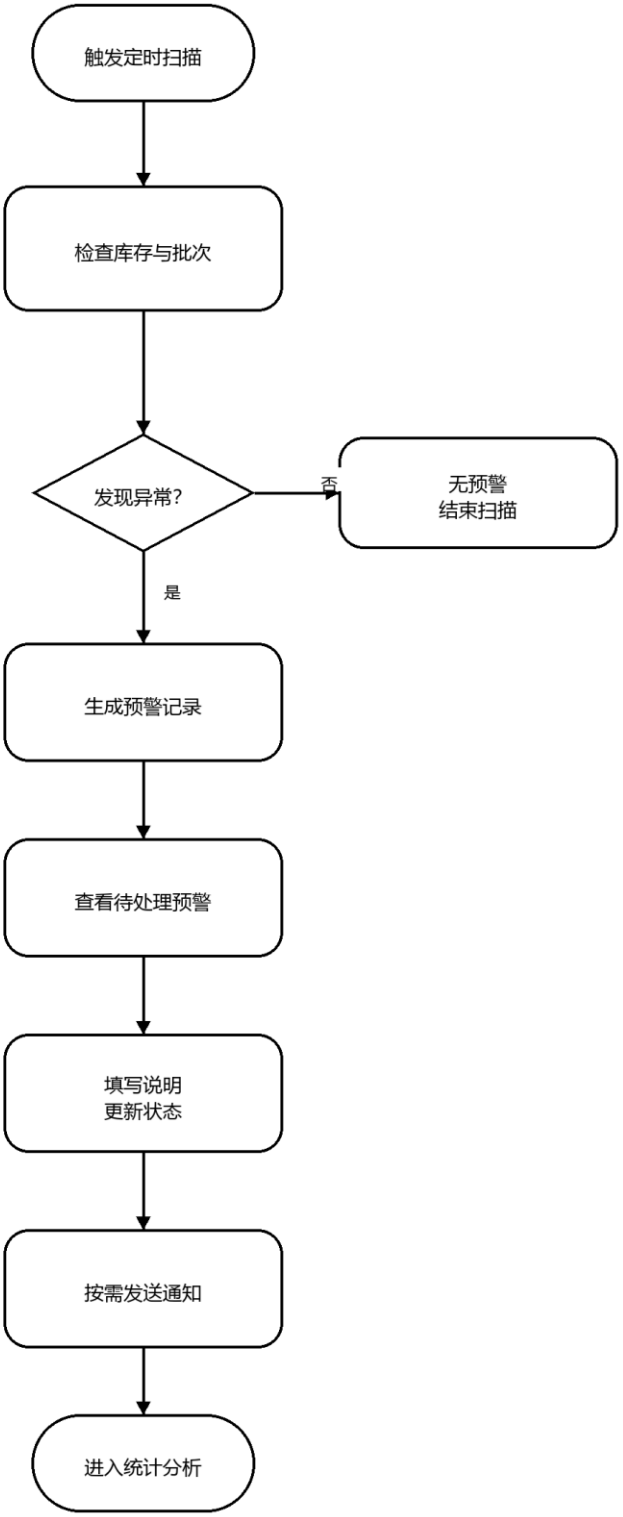
触发定时扫描
检查库存与批次
发现异常?
否
无预警
结束扫描
是
生成预警记录
查看待处理预警
填写说明
更新状态
按需发送通知
进入统计分析

图 3-3 预警处置流程图

4 系统总体设计

4.1 系统总体架构

从系统分层看，项目采用典型的 B/S 架构。浏览器作为统一访问入口，用户通过登录页进入系统后，根据角色获得不同菜单和页面；前端通过 HTTP/JSON 与后端接口交互；后端再按照控制层、服务层和数据访问层完成业务处理，并把结果写入 MySQL 数据库。该架构具有层次清晰、部署简洁和便于功能扩展的特点。

在后端内部，控制层主要负责参数接收和权限约束，服务层负责状态流转、库存计算、批次分配和日志记录，数据访问层由 MyBatis-Plus Mapper 与 JdbcTemplate 共同构成。控制层与服务层之间通过统一响应对象 `ApiResponse{code,message,data}` 进行对外封装，成功返回码固定为 `0`。这种统一返回机制使前端拦截器能够集中处理错误消息，减少页面端的重复判断。

在数据层面，系统并未采用高度拆分的微服务结构，而是使用单体后端配合模块化代码组织。结合系统业务规模，这种方案更利于业务联调和测试验证：认证、库存、申请、调拨、预警和统计虽然属于不同功能域，但都围绕同一套用户、角色、仓库和物资主数据展开，因此在单体架构中更容易维持事务语义和数据一致性。

系统总体架构如图 4-1 所示。

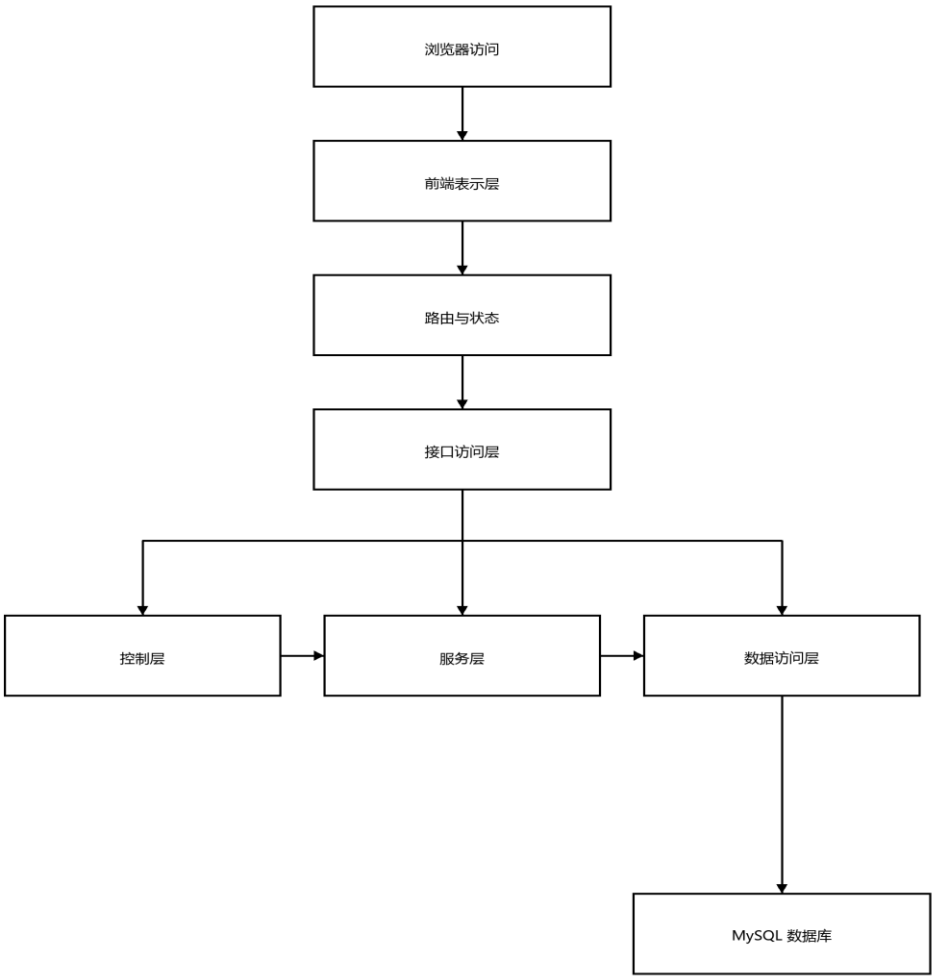

图 4-1 系统总体架构图

4.2 功能模块设计

根据前端路由、角色菜单和后端控制器，系统可以划分为八个相互关联的功能模块：认证与权限模块、基础数据模块、仓储库存模块、申领审批模块、调拨协同模块、预警与智能分析模块、统计分析模块、系统支撑模块。其中系统支撑模块又包括通知、事件、登录日志、操作日志和系统配置等功能，用于保障主业务的持续运行。

认证与权限模块负责用户登录、登出、刷新令牌、加载当前用户信息和动态菜单；基础数据模块负责用户、角色、部门、校区、仓库、库位、供应商、分类和物资档案维护；仓储库存模块负责入库、库存台账、批次追踪和出库执行；申领审批模块负责部门

侧申领单创建、提交、审批和签收；调拨协同模块负责跨仓调拨推荐、审批、执行和签收。这样的划分既贴合代码目录，也符合校园物资业务从“主数据准备”到“单据闭环执行”的实际顺序。

预警与智能分析模块、统计分析模块和系统支撑模块体现了项目的管理深度。本文所称“智能”主要指规则驱动和轻量算法驱动的自动感知、辅助决策与趋势分析能力：预警与智能分析模块负责定时预警、补货建议和 forecast 接口，统计分析模块负责图表与大屏聚合展示，系统支撑模块通过消息、日志与事件记录保证结果可追踪。整体来看，系统功能并非停留在 CRUD，而是围绕校园物资“计划、执行、监督、分析”的完整闭环展开。

系统功能模块划分如图 4-2 所示。

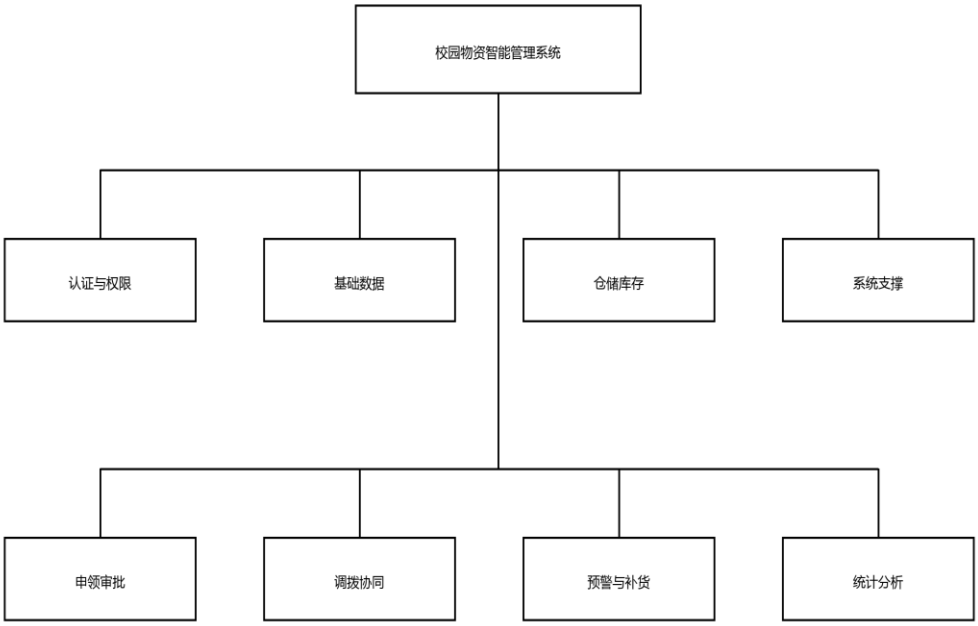

图 4-2 系统功能模块图

4.3 接口与安全设计

4.3.1 接口设计

接口设计不是抽象的 REST 口号，而是前后端协作的具体约束。当前项目统一以 /api 作为业务前缀，再按认证、库存、申领、调拨、预警、统计、事件和消息等功能域拆分接口，这一结构已经能从控制器分布和前端请求封装中直接看到。

更关键的是统一响应外壳。frontend/src/api/http.js 的拦截逻辑表明，所有业务接口都围绕 ApiResponse 的 code、message 和 data 三个字段组织返回结果，且前端把 code 等于 0 视为成功。这种一致性减少了页面侧的分支判断，也让异常处理可以回到同一出口。

4.3.2 认证与授权设计

认证链路由登录接口、令牌签发、刷新令牌续期和前端重放机制共同组成。JwtTokenProvider 负责生成和校验访问令牌，AuthService 负责登录、获取当前用户与菜单装配，http.js 则在 401 场景下触发刷新接口并重放原请求。这说明当前系统已经把登录态管理落实为一条完整链路，而不是简单地在浏览器里存一个长期有效字符串。

授权控制分布在两个层面。其一是 SecurityConfig 打开的方法级安全能力和控制器上的 PreAuthorize 表达式，其二是 AuthService.buildMenusByRole 返回的菜单集合。接口层和导航层一起工作后，系统管理员、仓库管理员、审批人和部门用户才真正表现出不同的页面与操作边界[20]。

4.3.3 安全与审计设计

在安全细节上，系统实现了密码加密、跨域控制、刷新令牌清理和审计留痕。用户密码采用 BCrypt 存储，刷新令牌以哈希形式写入 auth_refresh_token 表，并通过定时任务清理过期或撤销记录；登录日志和操作日志则为关键动作提供了追踪基础。

安全设计主要围绕访问控制、凭证管理和操作审计展开。密码加密、刷新令牌管理、跨域控制和日志留痕共同构成了系统的基础安全保障。高并发防护、网关治理和生产级安全运营仍需要结合专门测试和部署环境进一步完善。

4.4 智能能力设计

为保证“智能”概念在系统设计中的含义一致，表 4-1 对系统已实现的智能相关能力、输入数据、规则来源和适用范围进行归纳。

表 4-1 智能能力与实现方式对照表

能力名称	输入数据	规则/算法	输出结果	实现位置	适用范围
自动预警扫描	inventory、inventory_batch、stock_out_item、material_info.safety_stock	五类规则扫描 + 未处理预警去重	warning_record、预警列表与处理入口	WarningService.scan()；/api/warning/scan；WarningView	采用固定阈值与时间窗口，适用于规则驱动的库存风险识别
调拨推荐	targetCampus、materialId、qty、校区图、inventory.current_qty	Dijkstra 最短路径 + currentQty>=qty 过滤 + 距离升序排序	候选来源仓列表	TransferService.recommendTransfer()；DijkstraUtil；/api/transfer/recommend；TransferView	适用于候选来源仓排序，不涉及全局物流调度优化
补货建议与月度预测	近 30 天出库量、近 6 个月出库历史、safetyStock、guaranteeDays	日均消耗估算 + 月度均值预测	suggestQty、targetStock、forecast	SmartService；SmartController；/api/smart/*	通过服务层与接口提供预测和建议结果，适用于历史出库数据充分的场景
按效期优先批次执行	inventory_batch.expire_date、remain_qty	FEFO：expireDate ASC, id ASC	批次推荐顺序与实际扣减顺序	InventoryService.recommendOutbound()/stockOut()	属于批次执行规则优化，适用于按效期优先的出库场景

5 数据库设计

5.1 数据库设计原则

数据库设计以“主数据统一、业务单据分层、审计信息独立”为基本原则。主数据层负责保存组织结构、角色、用户、仓库、物资和供应商等静态或低频变化信息；业务单据层负责记录申领、出入库、调拨和预警；审计与支撑层负责保存日志、通知和事件等辅助信息。这样的分层方式可以避免把所有业务语义压缩到少数宽表中，便于后续统计查询和模块拆分。

从字段设计上看，多数业务表都包含 `deleted`、`version`、`created_at` 和 `updated_at` 字段，体现了逻辑删除、乐观锁预留和时间追踪的统一思想。对于库存、批次、单据、预警和日志等需要长期保留历史记录的场景，逻辑删除比物理删除更适合审计与回溯；统一时间字段则为统计分析、预警扫描和测试数据初始化提供了时间维度基础。

索引设计主要围绕业务查询入口展开。例如 `inventory` 表在物资与仓库上建立联合唯一约束，确保同一物资在同一仓库只有一条总库存记录；`inventory_batch` 表在到期日期上建立索引，便于按批次进行临期查询；申领单、调拨单、预警和日志等表也围绕状态、人员或时间字段建立索引，以支持列表页和统计查询的主要过滤条件。

5.2 实体关系设计

从实体关系角度看，本系统可以拆分为三组核心子域。第一组是 RBAC 与组织子域，包括部门、角色、用户、刷新令牌和日志，它决定了谁可以访问系统、可以看到哪些菜单以及关键操作由谁执行。第二组是库存与批次子域，包括分类、物资档案、仓库、库存总表、库存批次和出入库记录，它决定了库存总量与批次明细如何被维护。第三组是业务单据与支撑子域，包括申领单、调拨单、预警记录、通知和事件，它负责把具体业务动作串联为闭环。

这些子域之间存在清晰的主外键语义。部门用户与审批人都来自 `sys_user`，而 `sys_user` 又通过 `dept_id` 和 `role_id` 关联组织结构；库存和批次通过 `material_id` 与 `warehouse_id` 同时指向物资和仓库；申领单、调拨单和预警记录则分别通过用户、

部门、仓库和物资标识与主数据连接。虽然 SQL 定义没有在所有表上显式声明外键约束，但从字段命名和服务调用看，实体关系是稳定且明确的。

RBAC 与组织实体关系如图 5-1 所示。

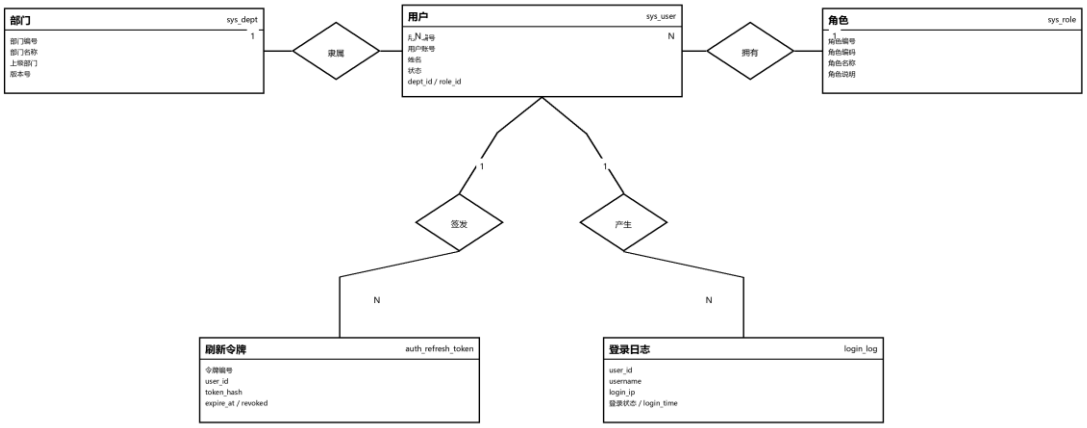

图 5-1 RBAC 与组织 E-R 图

库存与批次实体关系如图 5-2 所示。

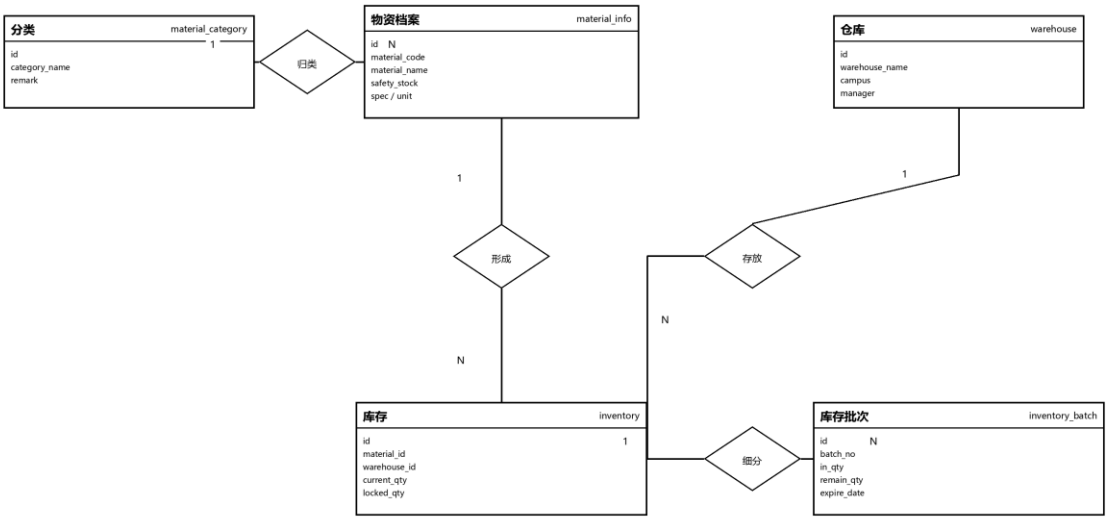

图 5-2 库存与批次 E-R 图

业务单据、预警与通知实体关系如图 5-3 所示。

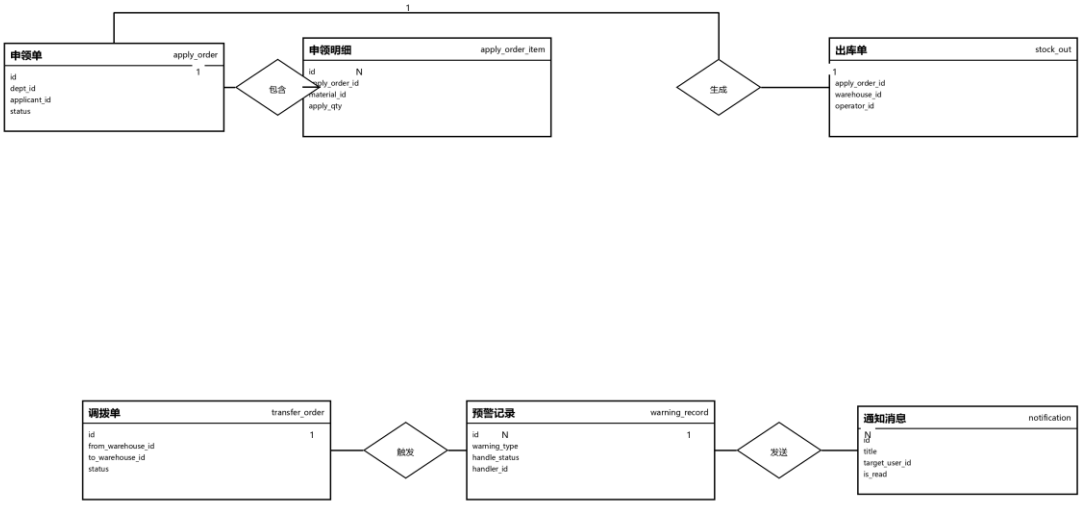

图 5-3 业务单据、预警与通知 E-R 图

各关键数据表字段设计如表 5-1 至表 5-12 所示。

5.3 关键数据表设计

关键数据表设计需要兼顾主数据维护、库存批次追踪、业务单据流转和审计追踪等需求。本节选取系统中的 12 张关键表进行详细说明，重点描述字段含义、约束关系及其在业务流程中的作用。

表 5-1 用户信息表（sys_user）

字段名	数据类型	长度	约束	注释描述
id	BIGINT	-	主键，自增	主键标识
username	VARCHAR	50	唯一，非空	登录账号
password	VARCHAR	255	非空	BCrypt 密码串
real_name	VARCHAR	100	非空	姓名
dept_id	BIGINT	-	可空	关联部门
role_id	BIGINT	-	可空	关联角色

字段名	数据类型	长度	约束	注释描述
status	TINYINT	-	非空，默认 1	当前状态标记
deleted	TINYINT	-	非空，默认 0	逻辑删除标记
version	INT	-	非空，默认 0	版本号
created_at	DATETIME	-	非空，默认当前时间	创建时间
updated_at	DATETIME	-	非空，默认当前时间	更新时间

表 5-2 物资档案表（material_info）

字段名	数据类型	长度	约束	注释描述
id	BIGINT	-	主键，自增	主键标识
material_code	VARCHAR	50	唯一，非空	物资编码
material_name	VARCHAR	100	非空	物资名称
category_id	BIGINT	-	可空	所属分类
spec	VARCHAR	100	可空	规格型号
unit	VARCHAR	20	可空	计量单位
safety_stock	INT	-	非空，默认 0	安全库存阈值
shelf_life_days	INT	-	可空	保质期天数
supplier	VARCHAR	255	可空	供应商名称
unit_price	DECIMAL	10,2	可空	单价
remark	VARCHAR	255	可空	备注信息
deleted	TINYINT	-	非空，默认 0	逻辑删除标记
version	INT	-	非空，默认 0	版本号

字段名	数据类型	长度	约束	注释描述
created_at	DATETIME	-	非空，默认当前时间	创建时间
updated_at	DATETIME	-	非空，默认当前时间	更新时间

表 5-3 仓库信息表（warehouse）

字段名	数据类型	长度	约束	注释描述
id	BIGINT	-	主键，自增	主键标识
warehouse_name	VARCHAR	100	非空	仓库名称
campus	VARCHAR	100	可空	所属校区
address	VARCHAR	255	可空	地址
manager	VARCHAR	100	可空	负责人
deleted	TINYINT	-	非空，默认 0	逻辑删除标记
version	INT	-	非空，默认 0	版本号
created_at	DATETIME	-	非空，默认当前时间	创建时间
updated_at	DATETIME	-	非空，默认当前时间	更新时间

表 5-4 库存总表（inventory）

字段名	数据类型	长度	约束	注释描述
id	BIGINT	-	主键，自增	主键标识
material_id	BIGINT	-	非空	关联物资
warehouse_id	BIGINT	-	非空	关联仓库
current_qty	INT	-	非空，默认 0	当前库存数量

字段名	数据类型	长度	约束	注释描述
locked_qty	INT	-	非空，默认 0	锁定数量
deleted	TINYINT	-	非空，默认 0	逻辑删除标记
version	INT	-	非空，默认 0	版本号
created_at	DATETIME	-	非空，默认当前时间	创建时间
updated_at	DATETIME	-	非空，默认当前时间	更新时间

表 5-5 库存批次表（inventory_batch）

字段名	数据类型	长度	约束	注释描述
id	BIGINT	-	主键，自增	主键标识
material_id	BIGINT	-	非空	关联物资
warehouse_id	BIGINT	-	非空	关联仓库
batch_no	VARCHAR	100	非空	批次号
in_qty	INT	-	非空，默认 0	入库数量
remain_qty	INT	-	非空，默认 0	剩余数量
production_date	DATE	-	可空	生产日期
expire_date	DATE	-	可空	到期日期
deleted	TINYINT	-	非空，默认 0	逻辑删除标记
version	INT	-	非空，默认 0	版本号
created_at	DATETIME	-	非空，默认当前时间	创建时间

字段名	数据类型	长度	约束	注释描述
updated_at	DATETIME	-	非空，默认当前时间	更新时间

表 5-6 申领单主表（apply_order）

字段名	数据类型	长度	约束	注释描述
id	BIGINT	-	主键，自增	主键标识
dept_id	BIGINT	-	非空	关联部门
applicant_id	BIGINT	-	非空	申请人
urgency_level	INT	-	非空，默认 0	紧急等级
status	VARCHAR	30	非空	当前状态标记
reason	VARCHAR	255	可空	业务原因
scenario	VARCHAR	255	可空	使用场景
fast_track	TINYINT	-	非空，默认 0	快速审批标记
approver_id	BIGINT	-	可空	审批人
approve_remark	VARCHAR	255	可空	审批意见
approve_time	DATETIME	-	可空	审批时间
deleted	TINYINT	-	非空，默认 0	逻辑删除标记
version	INT	-	非空，默认 0	版本号
created_at	DATETIME	-	非空，默认当前时间	创建时间
updated_at	DATETIME	-	非空，默认当前时间	更新时间

表 5-7 申领单明细表（apply_order_item）

字段名	数据类型	长度	约束	注释描述
id	BIGINT	-	主键，自增	主键标识
apply_order_id	BIGINT	-	非空	关联申领单
material_id	BIGINT	-	非空	关联物资
apply_qty	INT	-	非空	申领数量
actual_qty	INT	-	非空，默认 0	实际出库数量
deleted	TINYINT	-	非空，默认 0	逻辑删除标记
version	INT	-	非空，默认 0	版本号
created_at	DATETIME	-	非空，默认当前时间	创建时间
updated_at	DATETIME	-	非空，默认当前时间	更新时间

表 5-8 出库单主表（stock_out）

字段名	数据类型	长度	约束	注释描述
id	BIGINT	-	主键，自增	主键标识
apply_order_id	BIGINT	-	可空	关联申领单
warehouse_id	BIGINT	-	非空	关联仓库
operator_id	BIGINT	-	可空	操作人
remark	VARCHAR	255	可空	备注信息
deleted	TINYINT	-	非空，默认 0	逻辑删除标记
version	INT	-	非空，默认 0	版本号
created_at	DATETIME	-	非空，默认当前时间	创建时间

字段名	数据类型	长度	约束	注释描述
updated_at	DATETIME	-	非空，默认当前时间	更新时间

表 5-9 调拨单主表（transfer_order）

字段名	数据类型	长度	约束	注释描述
id	BIGINT	-	主键，自增	主键标识
from_warehouse_id	BIGINT	-	非空	调出仓库
to_warehouse_id	BIGINT	-	非空	调入仓库
status	VARCHAR	30	非空	当前状态标记
reason	VARCHAR	255	可空	业务原因
applicant_id	BIGINT	-	可空	申请人
approver_id	BIGINT	-	可空	审批人
approve_remark	VARCHAR	255	可空	审批意见
approve_time	DATETIME	-	可空	审批时间
deleted	TINYINT	-	非空，默认 0	逻辑删除标记
version	INT	-	非空，默认 0	版本号
created_at	DATETIME	-	非空，默认当前时间	创建时间
updated_at	DATETIME	-	非空，默认当前时间	更新时间

表 5-10 预警记录表（warning_record）

字段名	数据类型	长度	约束	注释描述
id	BIGINT	-	主键，自增	主键标识

字段名	数据类型	长度	约束	注释描述
warning_type	VARCHAR	50	非空	预警类型
material_id	BIGINT	-	可空	关联物资
warehouse_id	BIGINT	-	可空	关联仓库
content	VARCHAR	255	非空	内容描述
handle_status	VARCHAR	30	非空，默认 UNHANDLED	处理状态
handler_id	BIGINT	-	可空	处理人
handle_remark	VARCHAR	255	可空	处理说明
deleted	TINYINT	-	非空，默认 0	逻辑删除标记
version	INT	-	非空，默认 0	版本号
created_at	DATETIME	-	非空，默认当前时间	创建时间
updated_at	DATETIME	-	非空，默认当前时间	更新时间

表 5-11 通知消息表（notification）

字段名	数据类型	长度	约束	注释描述
id	BIGINT	-	主键，自增	主键标识
title	VARCHAR	200	非空	标题
content	VARCHAR	500	非空	内容描述
msg_type	VARCHAR	50	非空，默认 SYSTEM	消息类型
target_user_id	BIGINT	-	可空	目标用户

字段名	数据类型	长度	约束	注释描述
is_read	TINYINT	-	非空，默认 0	是否已读
biz_type	VARCHAR	50	可空	业务类型
biz_id	BIGINT	-	可空	业务编号
deleted	TINYINT	-	非空，默认 0	逻辑删除标记
version	INT	-	非空，默认 0	版本号
created_at	DATETIME	-	非空，默认当前时间	创建时间
updated_at	DATETIME	-	非空，默认当前时间	更新时间

表 5-12 登录日志表（login_log）

字段名	数据类型	长度	约束	注释描述
id	BIGINT	-	主键，自增	主键标识
user_id	BIGINT	-	可空	关联用户
username	VARCHAR	50	可空	登录账号
login_ip	VARCHAR	100	可空	登录 IP
login_status	VARCHAR	20	非空，默认 SUCCESS	登录状态
login_time	DATETIME	-	非空，默认当前时间	登录时间
user_agent	VARCHAR	500	可空	客户端标识
remark	VARCHAR	255	可空	备注信息
deleted	TINYINT	-	非空，默认 0	逻辑删除标记

字段名	数据类型	长度	约束	注释描述
version	INT	-	非空，默认 0	版本号
created_at	DATETIME	-	非空，默认当前时间	创建时间
updated_at	DATETIME	-	非空，默认当前时间	更新时间

6 系统详细设计与实现

6.1 开发环境与运行环境

系统使用前后端分离开发方式。后端基于 Maven 构建，依赖由 `backend/pom.xml` 管理；前端基于 Node.js 与 Vite 构建，依赖由 `frontend/package.json` 管理。测试环境通过 `backend/src/test/resources/application-test.yml` 将数据源切换为 H2 内存数据库，以便在不依赖外部 MySQL 服务的前提下执行自动化测试。

从实现角度看，开发环境与运行环境并未刻意追求复杂化。系统在 JDK 17、Maven 3.8+、Node.js 18+ 和 MySQL 8 环境下即可完成开发、测试和构建；前端通过 Vite 提供开发服务器与生产打包，后端通过 Spring Boot 提供接口服务与调度任务。这种环境配置符合课程设计项目“小而完整”的特点，有利于完成本地复现与功能验证。

系统开发与运行环境如表 6-1 所示。

表 6-1 系统开发与运行环境

项目	版本/配置	说明
JDK	17	后端运行环境，`pom.xml` 中声明 `java.version=17`
Maven	3.8+	后端依赖管理与打包工具
Spring Boot	3.3.5	后端基础框架
MyBatis-Plus	3.5.7	ORM 与通用数据访问框架
Node.js	18+	前端构建环境
Vue	3.5.29	前端核心框架，本次 `npm ls` 可见已安装版本
Vite	6.4.1	前端开发与构建工具，本次构建输出显示版本
Element Plus	2.13.5	前端组件库，本次 `npm ls` 可见已安装版本
ECharts	5.6.0	统计图表组件
MySQL	8.0	业务数据持久化存储
H2	内存模式	测试阶段数据源

6.2 认证授权与登录态管理实现

认证实现从 /api/auth/login 进入，但关键并不只是“用户输入账号密码”。前端 auth store 在登录成功后会立即拉取当前用户资料与动态菜单，说明登录动作和角色可见范围是一次连续初始化过程。对论文而言，这比单独介绍某个登录表单更能说明系统的真实运行方式。

登录态维持主要依赖 frontend/src/api/http.js 的请求与响应拦截器。请求阶段统一附加 Bearer Token，响应阶段先检查 ApiResponse 的 code，再在收到 401 时进入刷新流程。脚本中维护的等待队列可以避免多个并发请求重复刷新令牌。

后端与之配合的是 JwtTokenProvider、AuthService 和 auth_refresh_token 表。访问令牌负责短期身份识别，刷新令牌负责续期与撤销，清理任务再把过期记录回收掉。这样形成的不是概念上的 JWT 介绍，而是一条已经写进代码和数据表的登录、续期、退出链路。

认证授权与登录态维持的请求处理链路如图 6-1 所示。

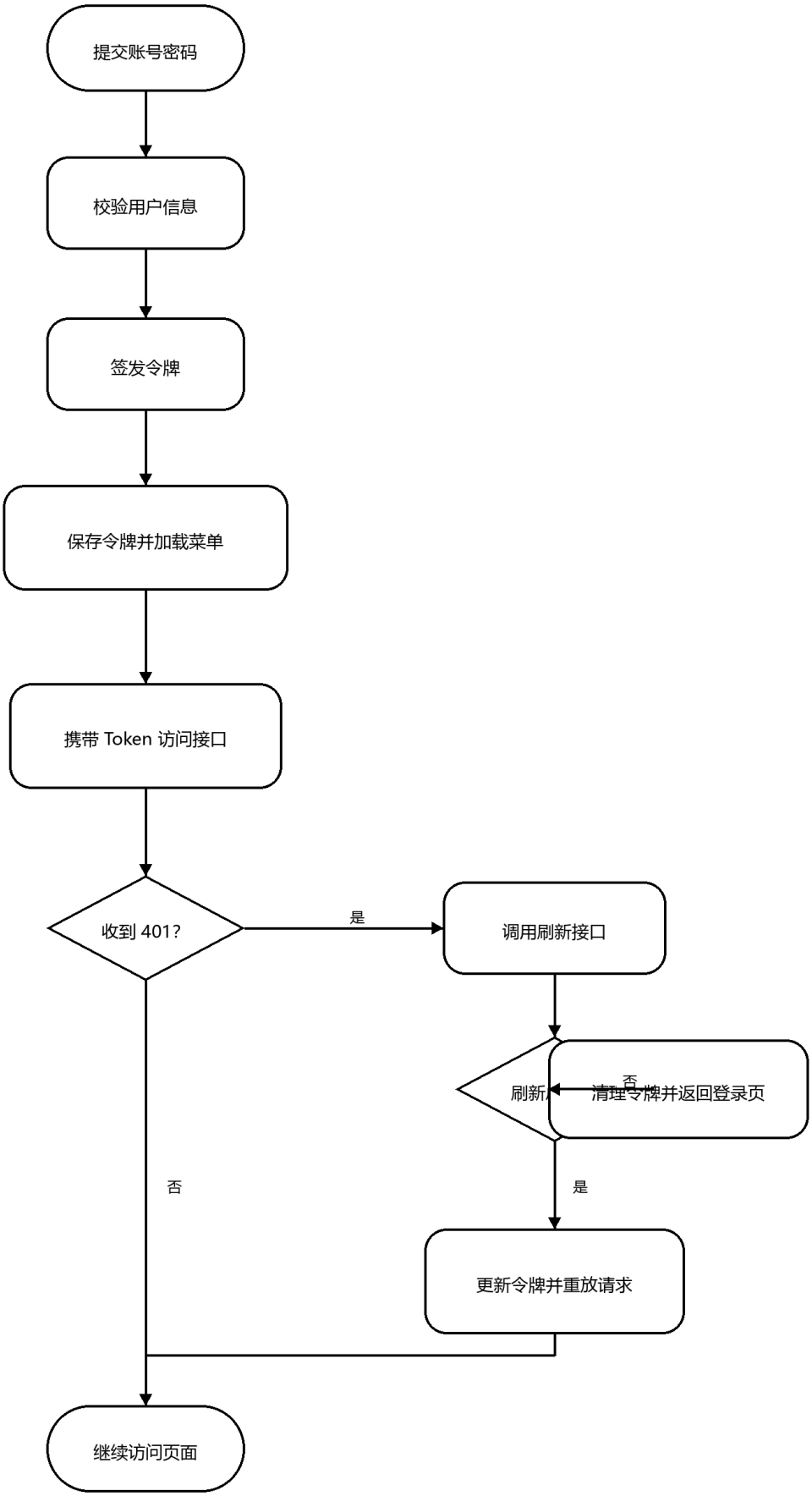
提交账号密码
校验用户信息
签发令牌
保存令牌并加载菜单
携带 Token 访问接口
收到 401？
是
调用刷新接口
刷新
否
清理令牌并返回登录页
是
更新令牌并重放请求
否
继续访问页面

图 6-1 登录认证与令牌续签流程图

系统登录入口与统一认证界面如图 6-2 所示。

图 6-2 系统登录界面

6.3 基础数据管理实现

基础数据管理模块覆盖用户、角色、部门、校区、仓库、库位、供应商、分类和物资档案等对象。前端针对这些对象分别提供列表、查询、新增、编辑和删除页面，后端则以相对统一的控制器和服务模式组织 CRUD 逻辑。虽然这些页面的交互形式类似，但在系统中承担的作用并不相同：部门和角色决定权限边界，仓库与库位决定库存归属，供应商和物资档案则决定入库、预警和补货分析的主数据来源。

物资档案模块是基础数据中的关键节点。物资编码、名称、分类、规格、单位、安全库存、保质期、供应商和单价等信息不仅用于列表展示，还会被库存预警、出库扣减和补货建议等后续逻辑直接使用。因此，物资档案模块的实现重点不在于界面表单本身，而在于通过统一主数据避免“同一物资多种命名、不同仓库重复维护”的问题。

基础数据模块还承担初始化业务场景的作用。`seed.sql` 为管理员、仓库管理员、审批人和部门用户准备了基础账号和样例数据，使系统在开发与测试阶段能够快速进入可操作状态，并具备支撑完整业务链路的初始数据环境。

6.4 库存、批次、出入库实现

库存管理模块的核心是同时维护库存总量与批次明细。入库时，系统不仅更新 `inventory` 表中的 `current_qty`，还在 `inventory_batch` 中写入批次号、入库量、剩余量、生产日期和到期日期，从而保证总库存和批次账保持一致。出库时则反向根据批次顺序扣减剩余量，再同步更新总库存。这样设计的直接好处，是可以在总量统计与批次追溯之间取得平衡。

`InventoryService` 的实现体现了基于有效期的规则型执行智能。系统查询指定物资在某仓库下所有剩余数量大于 0 且未过期的批次，按 `expire_date` 和主键升序排列后依次扣减，这一过程对应 FEFO（first expired, first out）策略。该策略既能优先消耗临近到期物资、降低损耗风险，也让批次选择过程保持可解释。

出入库模块还负责把库存变化与单据状态关联起来。入库单负责建立新批次并增加库存，出库单则通常与申领单关联，在完成拣货后回写申领明细实际数量并推动申领单进入 `OUTBOUND` 状态。也就是说，库存模块并不是孤立计算数量变化，而是在业务上下文中完成库存扣增、批次更新和状态推进的联动。

系统在出库前引入申领库存预占机制。ApplyService.submit() 提交申领单时会调用 reserveForApplyOrder() 锁定可满足单据的仓库库存，库存总表中的 lockedQty 用于表示已被未完成申领占用的数量；驳回、签收或实际出库后再释放对应锁定量。这一机制解决的是并发一致性与可靠性问题，应归类为库存保护，而不是智能推荐算法。

6.5 申领审批与调拨实现

6.5.1 申领审批实现

申领审批实现围绕 `ApplyService` 展开。创建申领单时，系统先插入主表 `apply_order`，初始化状态为 `DRAFT`，再批量插入明细表 `apply_order_item`。提交时若单据仍处于草稿状态，则进一步根据 `urgency_level` 判断流转分支：普通单据进入 `SUBMITTED` 等待审批，紧急等级大于等于 2 的单据则直接置为 `APPROVED`，同

时记录审批人、审批意见和审批时间。这一逻辑比传统论文中常见的“提交后统一待审”更贴近真实业务约束。

审批通过、驳回和签收等动作都由明确的方法处理，并通过 `OrderStatus` 常量约束状态机边界。例如，只有 `SUBMITTED` 状态的单据可以被审批或驳回，只有 `OUTBOUND` 状态的单据才允许签收。每个关键动作完成后，系统都会调用 `OperationLogService` 写入操作日志，从而形成业务动作与审计记录的同步留痕。

申领审批模块的实现价值在于，它把角色协同、单据明细、状态机和库存执行串成了一条完整链路。部门用户关注的是“能否快速发起申请”，审批人关注的是“是否符合领用条件”，仓库管理员关注的是“如何准确出库并回写数量”，而系统通过主表、明细表和日志表把这些差异化需求统一到同一套数据模型下。

部门用户发起物资申请时，可以在申领页面直接录入原因、场景和物资明细，运行界面如图 6-3 所示。

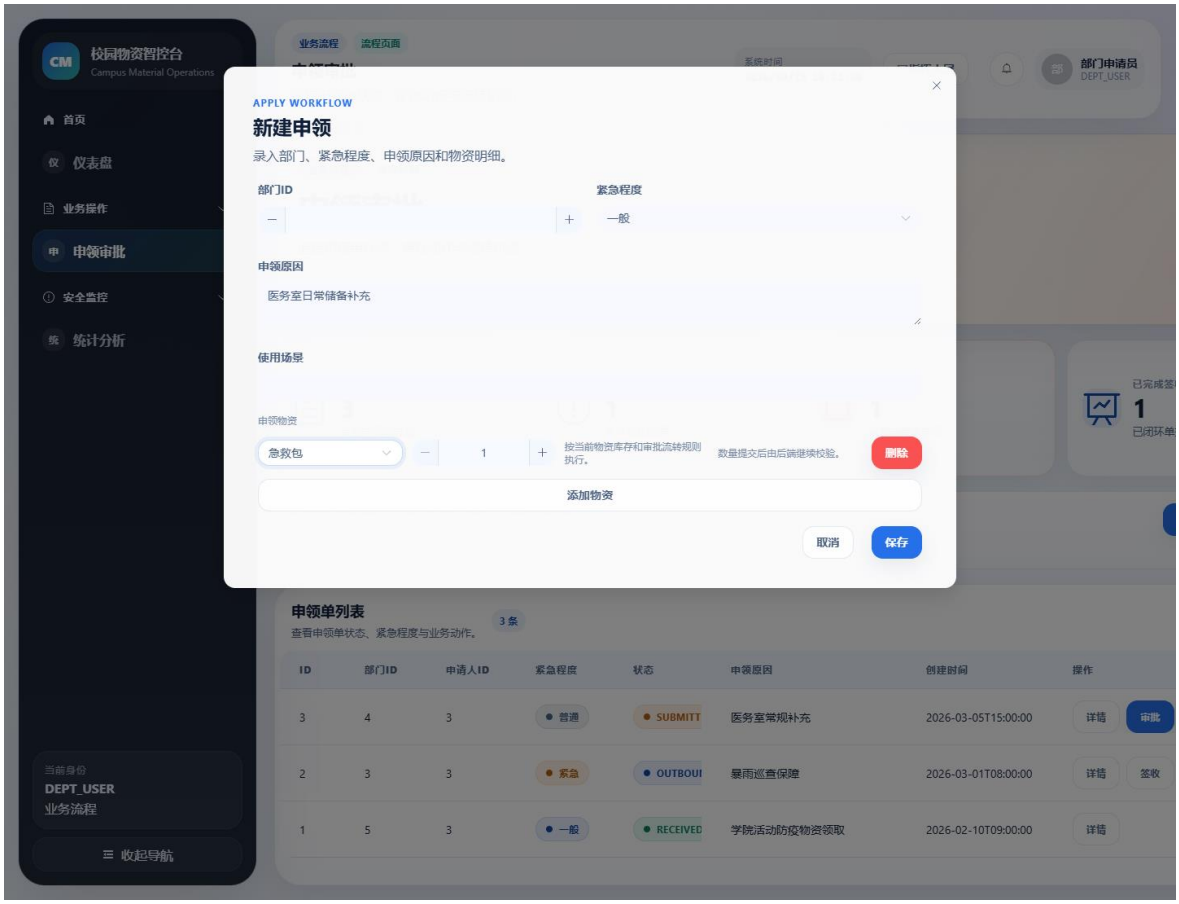

图 6-3 部门用户申领界面

审批人与仓库管理员分别在审批列表和出库页面完成业务闭环，组合运行界面如图 6-4 所示。

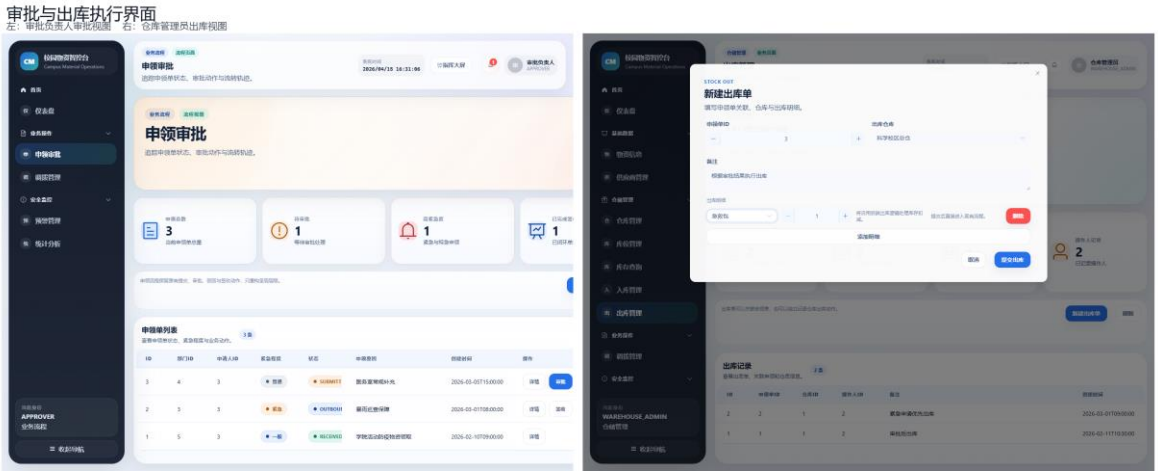

图 6-4 审批与出库执行界面

6.5.2 调拨实现与推荐机制

调拨实现与申领类似，也采用主表加明细表模式，但其复杂度更高。`TransferService.create()` 在校验调出仓与调入仓不能相同后生成 `DRAFT` 状态的调拨主单和多条明细；提交、审批、驳回、执行与签收则由独立方法推进状态。执行阶段系统需要同时处理来源仓批次扣减、目标仓镜像批次写入和两端库存汇总更新，因此调拨不是简单改一个库存数字，而是一次受事务约束的跨仓迁移。

推荐逻辑则由 `TransferService.recommendTransfer()` 与 `DijkstraUtil.calculateShortestPaths()` 共同完成。`DijkstraUtil` 预置了“科学校区、东风校区、龙子湖校区、中心物资周转区、附属医院急救仓”等节点及其边权，服务层先以目标校区为源点计算到各节点的最短距离，再把距离结果映射到仓库所属校区。

在距离计算完成后，服务层继续从 `inventory` 表中筛选指定 `materialId` 且 `currentQty >= qty` 的候选库存记录，批量加载对应仓库信息，并按 `distance` 升序组织返回结果。因而这里的“智能推荐”本质上是“最短路径 + 库存可用量约束”的候选来源仓排序，能够为仓库管理员提供可解释的辅助决策。

这一机制的适用边界是候选仓筛选和排序，不涉及车辆路径规划、运输时效预测或复杂物流调度优化。该设计的价值在于轻量、可解释，并且与校园多校区物资调拨场景相匹配。

调拨执行与推荐流程如图 6-5 所示。

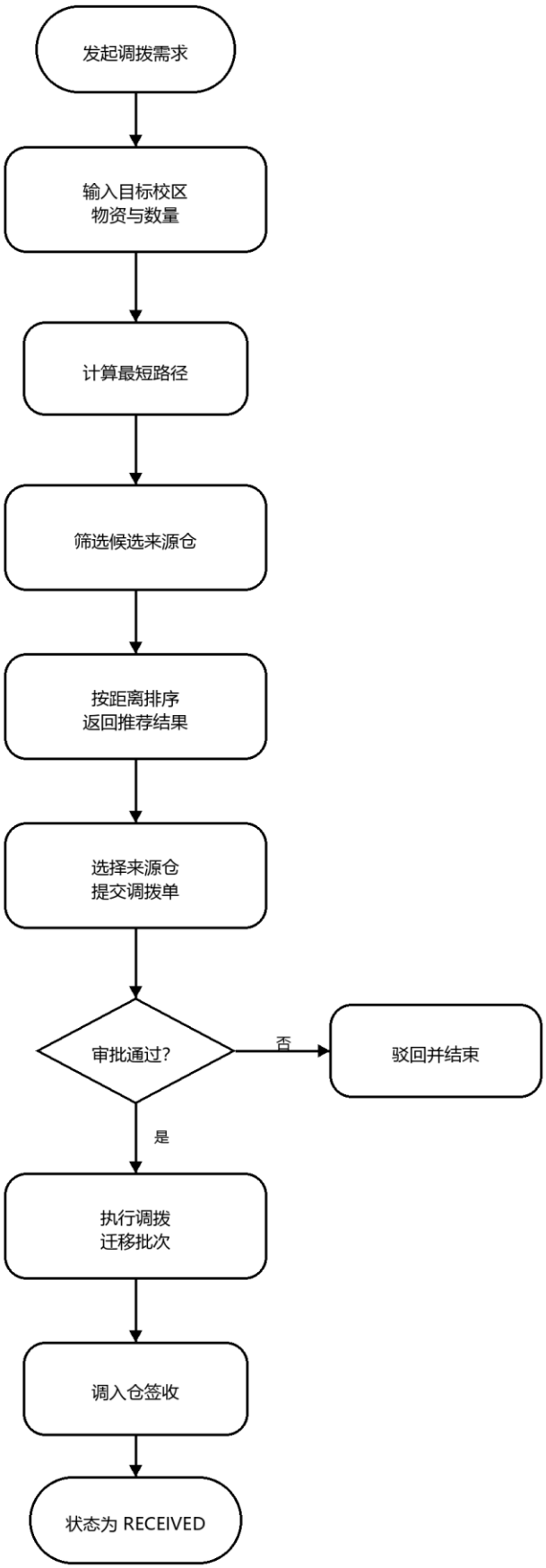
发起调拨需求
输入目标校区
物资与数量
计算最短路径
筛选候选来源仓
按距离排序
返回推荐结果
选择来源仓
提交调拨单
审批通过?
否
驳回并结束
是
执行调拨
迁移批次
调入仓签收
状态为 RECEIVED

图 6-5 调拨执行与推荐流程图

仓库管理员在调拨页面可直接发起跨仓调拨，并调用推荐结果辅助选择来源仓库，运行界面如图 6-6 所示。

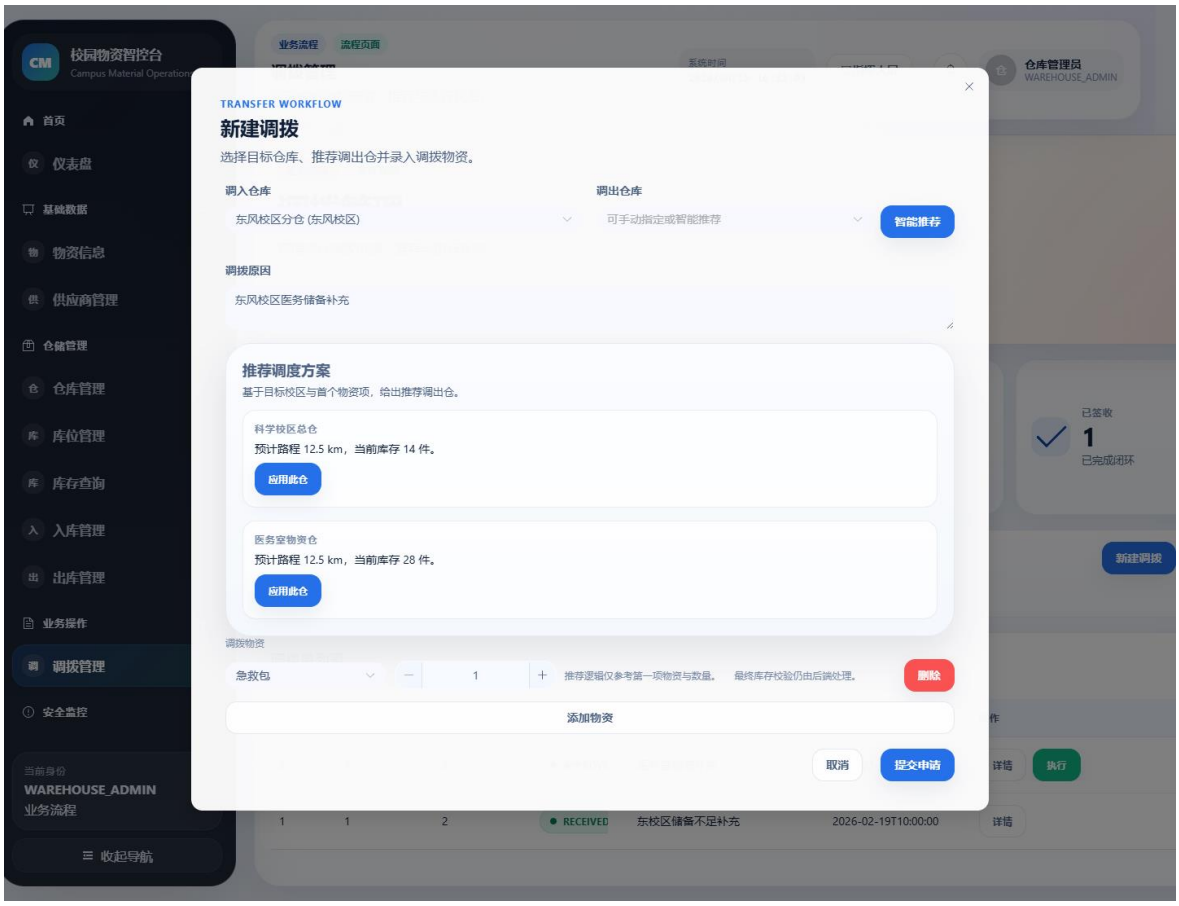

图 6-6 调拨推荐界面

6.6 预警、补货建议与统计分析实现

预警模块由 `WarningService` 负责实现，核心入口是带有 `@Scheduled(cron = "0 0/30 * * * ?")` 注解的 `scan()` 方法。系统每 30 分钟扫描一次库存与批次数据，分别执行低库存、库存积压、临期、过期和异常领用 5 类检测，并在不存在相同未处理预警时创建新的 `warning_record`。这一定时扫描机制让风险发现从“人工查看列表”前移为“系统主动发现异常”。

5 类规则的判定条件都是固定且可复核的：低库存对应 `currentQty < safetyStock`，积压对应 `currentQty > safetyStock * 3`，临期对应 `today <= expireDate <= today + 30 days`，过期对应 `expireDate < today`，异常领用对应近 7 天出库量大于近 30 天折算

周均值的 `1.5` 倍。与此同时，`createWarningIfAbsent()` 还会按预警类型、物资、仓库和未处理状态进行去重，避免同一问题被重复刷屏。

智能分析模块由 `SmartService` 提供两个轻量能力。其一是 `forecast()`：读取近 6 个月出库历史，按月份汇总数量，计算月度均值，并按未来月份输出 `forecast` 结果；其二是 `replenishmentSuggestions()`：基于近 30 天出库量估算 `avgDailyUsage = outQty30 / 30.0`，再结合 `safetyStock` 和 `guaranteeDays` 计算 `targetStock = safetyStock + avgDailyUsage * guaranteeDays`，最后得到 `suggestQty = max(0, targetStock - currentStock)`。这种实现方式与文献中常见的移动平均思想保持一致，但工程上采用的是简单、可解释、可落地的平均值计算，而不是复杂预测模型[21]。

统计分析则由 `AnalyticsService` 通过聚合 SQL 实现。系统围绕概览指标、库存分布、月度出入库趋势、部门排行、效期统计和紧急领用趋势生成图表数据，并由统计分析页和大屏页负责展示。由于这些结果直接依赖业务表的真实记录，其价值不只是展示图表，更在于为管理员提供观察库存风险和消耗趋势的管理视角。

`SmartService` 的 `forecast` 与 `replenishmentSuggestions` 通过 `/api/smart/forecast` 和 `/api/smart/replenishment-suggestions` 接口对外提供。该模块基于历史出库记录进行月度均值预测和补货量估算，属于轻量规则与统计计算能力，未涉及复杂模型训练或自适应参数优化。

预警模块在列表中同时展示预警类型、处理状态和处置入口，真实运行界面如图 6-7 所示。

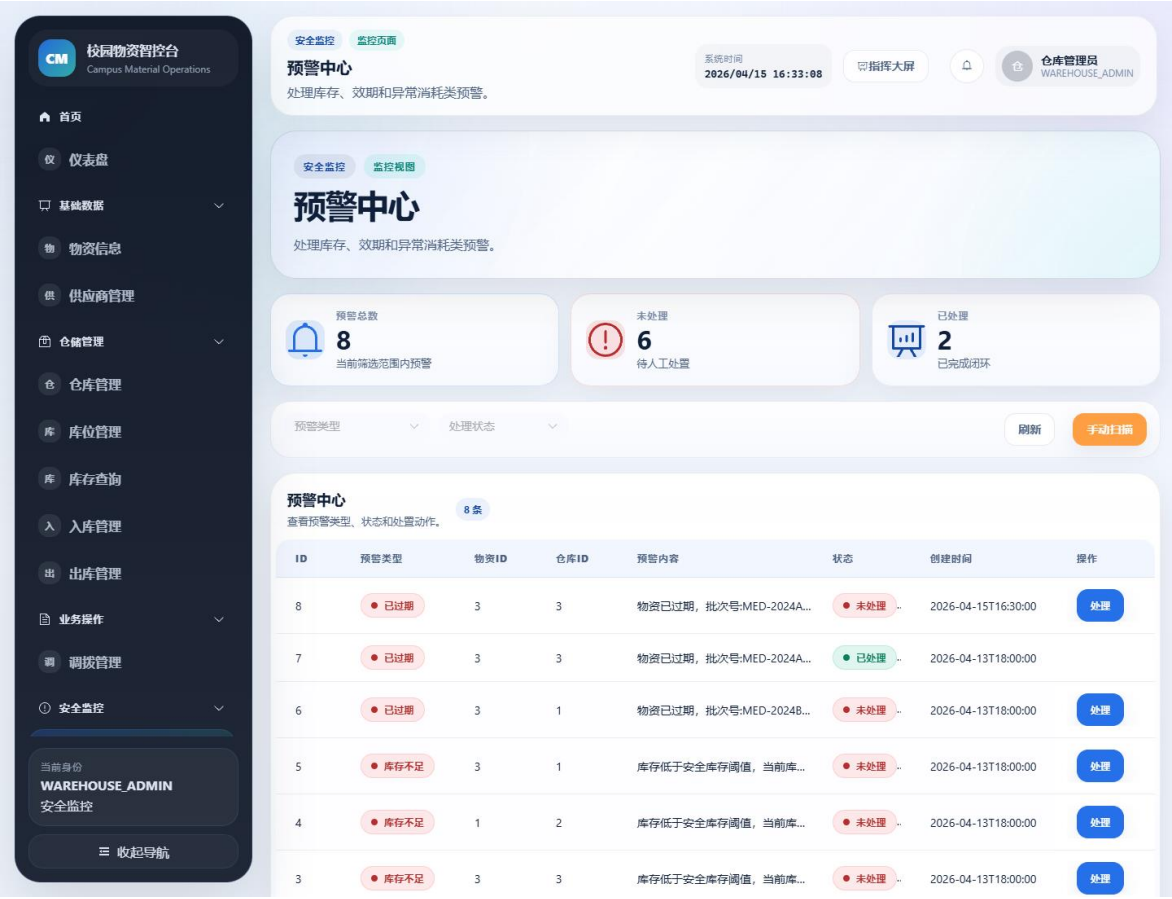

图 6-7 预警管理界面

统计分析模块通过图表卡片集中展示库存、趋势和排行结果，真实运行界面如图 6-8 所示。

图 6-8 统计分析界面

6.7 日志、通知与事件管理实现

日志、通知和事件管理模块承担的是“让业务过程可追踪”的职责。登录日志记录用户账号、登录 IP、登录结果、时间和客户端标识，操作日志记录操作人、业务模块、操作类型和明细内容，两者共同构成对关键行为的审计基础。由于申领、审批、出库、调拨、预警处理等操作都会写入日志，系统能够在后续排查中回溯“谁在什么时候做了什么”。

通知模块以 `notification` 表为中心，保存标题、内容、消息类型、目标用户、已读状态及关联业务信息。它的实现价值在于把业务结果从“页面即时提示”扩展为“可再次查看的消息记录”，从而适应校园场景中审批、预警、事件等信息需要跨时间查看的需求。部门用户、审批人和仓库管理员可以在自身角色页面中查看与自己相关的消息，提升协同效率。

事件管理模块进一步扩展了系统对校园保障场景的覆盖面。`event_record` 表可以记录事件标题、类型、等级、位置、上报人、处理结果和关闭时间，体现校园突发事件

与物资管理之间的联动关系。作为支撑模块，事件管理能够提升系统对校园综合保障业务的覆盖能力。

7 系统测试

7.1 测试环境与验证方法

本章从后端自动化测试、H2 测试配置、Maven 测试结果和前端生产构建结果四个方面说明系统验证情况。测试内容聚焦统一响应与异常处理、认证与令牌管理、申领业务、调拨业务、预警接口、集成主链路和前端基础交互。

后端测试环境由 backend/src/test/resources/application-test.yml 指向 H2，并结合测试资源目录中的 schema 初始化数据结构。这意味着关键业务逻辑可以在不依赖外部 MySQL 实例的情况下重复执行，测试结果也更容易在开发环境中复核。

测试过程中执行 `mvn -f backend/pom.xml test`、`npm --prefix frontend run build` 与 `npm --prefix frontend run test:unit -- --run`。三条命令分别得到后端 49 项测试通过、前端生产构建成功和前端 8 项单元测试通过的结果，能够反映系统核心模块的可执行性。

第 6 章中的界面图用于展示主要功能页面的运行形态，页面数据仅用于说明交互流程和模块入口，不作为性能或部署效果的评价依据。

7.2 自动化测试结果

从测试类分布看，自动化测试主要集中在系统主链路和高风险规则上，包括统一响应与异常处理、认证与令牌管理、申领业务、调拨业务、预警接口，以及围绕登录刷新、申领出库、调拨执行、分页与并发库存保护的集成主链路。该测试覆盖方式能够优先验证状态流转、权限校验和关键业务规则。

执行 Maven 测试后，后端共通过 12 个测试类、49 项测试；执行前端构建与单元测试后，Vite 生产构建成功，Vitest 共 3 个测试文件、8 项测试通过。这个结果说明当前版本在核心业务逻辑、配置加载和前端基础交互层面仍保持可执行状态。

后端自动化测试执行情况如表 7-1 所示。

表 7-1 后端自动化测试执行情况

测试类别	对应测试类/范围	验证重点	结果
统一响应与异常处理	ApiResponseTest、BizExceptionTest、GlobalExceptionHandlerTest	统一返回结构、异常封装与错误消息	执行通过
认证与令牌管理	AuthServiceTest、JwtTokenProviderTest、AuthRefreshTokenCleanupTaskTest	登录、JWT、刷新令牌清理与菜单装配	执行通过
申领业务	ApplyServiceTest、ApplyControllerTest	申领单创建、提交、审批和接口访问	执行通过
调拨业务	TransferServiceTest、TransferControllerTest	调拨单创建、审批、执行和推荐接口	执行通过
预警接口	WarningControllerTest	预警列表与处理接口	执行通过
集成主链路	CoreFlowIntegrationTest	登录/刷新、申领、调拨、预警、分页与并发库存保护	执行通过
总体结果	共 12 个测试类、49 项测试	核心模块可执行性与主链路一致性	全部通过

7.3 业务场景验证

除自动化测试外，系统还可以基于代码、路由和样例数据对典型业务场景进行复验说明。相关场景强调主链路已经具备清楚的入口、状态变化和落库对象，能够支撑后续功能演示与人工核查。

典型业务场景验证情况如表 7-2 所示。

结合系统实现，至少有三类智能相关场景已经形成闭环：仓库管理员创建调拨单并通过推荐接口获得候选来源仓；系统按固定周期生成 5 类预警并在预警页提供处置入口；`SmartService` 基于历史出库记录返回补货建议和月度预测结果，`AnalyticsService`

再通过图表页和大屏页提供管理视图。由于 `SmartService` 未设置独立精度评估数据集，本文仅从功能可用性和接口返回结果角度进行验证，不展开算法精度评价。

表 7-2 典型业务场景验证说明

业务场景	主要入口	验证对象	说明
普通申领审批与签收	/apply/list	ApplyService、InventoryService、OrderStatus	支持 DRAFT→SUBMITTED→APPROVED→OUTBOUND→RECEIVED 流转
紧急申领快速审批	/apply/list	ApplyService.submit()	urgency_level>=2 时直接审批通过并记录审批信息

业务场景	主要入口	验证对象	说明
跨仓调拨推荐与执行	/transfer/list	TransferService、DijkstraUtil、/api/transfer/recommend	支持按最短路径与可用库存筛选候选来源仓，并完成审批、批次迁移与签收
预警扫描与处理	/warning/list	WarningService.scan()、warning_record	支持五类预警生成、未处理去重与处理状态更新

业务场景	主要入口	验证对象	说明
补货建议与趋势分析	/api/smart/*、/analytics/charts、/bigscreen	SmartService、SmartController、AnalyticsService	支持近 6 个月均值预测和 30 天日均消耗估算
日志与通知追踪	/log/*、/notification/list	login_log、operation_log、notification	支持对关键业务动作的审计与消息留存

7.4 构建验证与问题边界

前端构建验证来自 `npm --prefix frontend run build` 的执行结果。构建成功说明系统中的 Vue 页面、路由、组件依赖和打包配置保持一致，未出现阻断交付的基础构建错误。

现有测试与构建结果可以证明系统在当前覆盖范围内具备可执行性和核心逻辑正确性，但不能代表高并发稳定性、生产网络部署效果和长期运维可靠性。相关能力仍需要在后续工作中通过压测、监控数据和生产运行日志进一步验证。

综上，测试结果能够说明系统主要业务链路在当前覆盖范围内保持可执行状态，满足本科毕业设计中对功能完整性和工程可验证性的基本要求。

结束语

本文围绕校园物资智能管理系统，对需求分析、总体设计、关键实现和测试验证进行了系统阐述。系统以校园物资管理业务为对象，形成了基础数据维护、库存批次管理、申领审批、跨仓调拨、预警处置、统计分析和日志通知等功能模块，并通过规则扫描、路径推荐、补货建议和效期优先策略体现智能化管理能力。

从现有实现看，系统已经完成了认证授权、基础数据维护、库存与批次管理、申领审批、调拨协同、预警扫描、补货建议、统计分析、通知消息、事件记录和日志审计等核心能力。其中“智能”并不是额外包装出来的概念，而是落实为四类可验证能力：风险主动发现、跨仓调拨辅助决策、补货测算与趋势预测、按效期优先的批次执行规则。

这些能力的共同特点是规则清晰、输入明确、输出可解释，能够与 `WarningService`、`TransferService.recommendTransfer`、`DijkstraUtil`、`SmartService` 和 `InventoryService` 等实现一一对应。它们提升的是校园物资系统的自动感知与辅助决策能力，而不是通过黑箱模型替代业务人员判断。

测试结果表明，系统后端 49 项测试通过，前端生产构建成功，前端 8 项单元测试通过。由此可以看出，系统在核心业务逻辑、基础交互和工程构建方面具备较好的可执行性。后续工作可进一步围绕模型化优化、历史数据积累、业务指标分析和生产级运维能力进行扩展。

致　　谢

时光荏苒，大学四年的学习生活即将结束，本科毕业设计也在不断修改与完善中顺利完成。回顾从选题、需求分析、系统设计、功能实现到论文撰写的整个过程，我不仅进一步巩固了软件工程专业知识，也提升了独立分析问题、解决问题和工程实践的能力。在此，我谨向所有给予我指导、帮助和支持的老师、同学及家人表示诚挚的感谢。

首先，衷心感谢我的指导老师。在毕业设计的选题确定、系统功能规划、技术路线选择、论文结构调整以及后期修改完善过程中，老师始终给予我耐心细致的指导。每当我在系统实现、论文撰写或格式规范方面遇到问题时，老师都能及时提出宝贵意见，使我对软件工程项目开发流程和本科毕业论文写作要求有了更加深入的认识。老师严谨的治学态度、认真负责的工作作风和精益求精的学术精神，也将对我今后的学习和工作产生积极影响。

其次，感谢软件学院各位任课老师在本科阶段对我的培养。通过系统学习 Java 程序设计、数据库原理、数据结构、操作系统、软件工程等课程，我逐步建立了较为完整的专业知识体系，也为本次毕业设计中后端开发、数据库设计、系统测试和论文撰写奠定了基础。各位老师在课堂教学和实践指导中的认真负责，使我受益匪浅。

同时，感谢同学和朋友们在毕业设计完成过程中给予我的帮助与鼓励。在系统调试、功能测试、资料整理和论文修改过程中，大家提出了许多有益建议，也帮助我发现并改进了部分不足之处。正是这些交流与支持，使我能够更加顺利地完成本次毕业设计。

最后，感谢我的家人一直以来对我的理解、支持与鼓励。你们在学习和生活中给予我的关心与陪伴，是我不断前进的重要动力。今后，我将继续保持踏实认真的态度，不断学习、不断进步，以更加积极的状态面对未来的学习和工作。

谨以此文，向所有关心、帮助和支持过我的人致以最诚挚的感谢。

参考文献

[1] 陈志敏, 何建国. 大数据背景下智慧校园建设与管理模式创新探讨[J]. 现代教育管理, 2020, 12(8): 45-51.

[2] 张强, 李红梅. 高校突发公共事件应急物资储备体系构建研究[J]. 中国安全生产科学技术, 2022, 18(3): 67-73.

[3] 李淼淼, 季节, 李坡, 等. 基于 SpringBoot 和 Vue 3 的移动学习管理系统的设计与实现[J]. 无线互联科技, 2026, 23(04): 81-86.

[4] 杨茜, 吴加莹. 基于 B/S 架构的高校教务管理系统设计与实现[J]. 电脑知识与技术, 2026, 22(06): 58-60,76.

[5] 崔靖茹, 文华, 刘宏磊, 等. 基于 Vue 和 SpringBoot 框架的高校信息化项目管理系统的设计与实现[J]. 现代信息科技, 2025, 9(22): 77-81.

[6] 薛楠楠, 张伟, 钟化雨, 等. 基于 SpringBoot+Vue 的事故报告数据库系统设计与实现[J]. 计算机仿真, 2025, 42(07): 456-462,570.

[7] 刘丹阳, 杨华, 张诗桐. 基于 Django 的个性化图书推荐管理系统设计与实现[J]. 山西电子技术, 2025(03): 64-66,80.

[8] 岑丹. 基于人工智能的图书馆信息管理系统设计与实现[J]. 吉林大学学报(信息科学版), 2025, 43(06): 1346-1351.

[9] 王伟, 刘明. 基于 Spring Boot 的后端架构设计与性能优化研究[J]. 计算机系统应用, 2021, 30(5): 102-109.

[10] Spring Team. Spring Boot Reference Documentation[EB/OL]. [2026-04-23]. https://docs.spring.io/spring-boot/docs/3.3.5/reference/html/.

[11] Vue Team. Vue.js Documentation[EB/OL]. [2026-04-23]. https://vuejs.org/guide/introduction.html.

[12] Vite Team. Vite Guide[EB/OL]. [2026-04-23]. https://vite.dev/guide/.

[13] Jones M, Bradley J, Sakimura N. JSON Web Token (JWT): RFC 7519[EB/OL]. [2026-04-23]. https://www.rfc-editor.org/rfc/rfc7519.

[14] Dijkstra E W. A note on two problems in connexion with graphs[J]. Numerische Mathematik, 1959, 1: 269-271.

[15] 刘强, 周波. MyBatis-Plus 企业级高可用开发指南[M]. 北京: 电子工业出版社, 2021: 89-105.

[16] 李东, 赵莉. Vue3+Vite 开发实战: 组合式 API 与工程化实践[M]. 北京: 人民邮电出版社, 2023: 155-180.

[17] 李晓东. MySQL 8.0 性能调优实战教程[M]. 北京: 清华大学出版社, 2022: 24-39.

[18] 赵玉明, 林晓. ECharts 在 Web 报表数据可视化中的应用分析[J]. 信息技术与信息化, 2021(4): 21-25.

[19] 杨俊. 新一代 Spring Security 原理与 JWT 无状态鉴权探索[D]. 武汉: 武汉科技大学, 2020.

[20] 冯绍彬, 周华. 基于角色的动态权限管理系统的应用设计[J]. 郑州轻工业大学学报(自然科学版), 2020, 35(4): 71-76.

[21] 孙阳, 陈洁. 一种基于移动平均预测模型的库存预警系统设计[J]. 物流工程与管理, 2022, 44(2): 33-38.

附录 1 关键接口与测试补充说明

附录内容用于补充正文中不宜展开的接口入口与测试范围说明，相关条目均来源于系统现有模块、控制器和测试类。

附表 1-1 关键业务接口补充说明

功能域	主要接口	对应模块	说明
认证授权	/api/auth/login、/api/auth/refresh	AuthController、AuthService	完成登录认证、令牌续签和当前用户信息加载
库存与申领	/api/inventory/*、/api/apply/*	InventoryService、ApplyService	支撑库存分页、入库出库、申领提交、审批和签收
调拨与预警	/api/transfer/*、/api/warning/*、/api/smart/*	TransferService、WarningService、SmartService	支撑调拨推荐、调拨执行、预警扫描和补货建议

附录测试范围包括后端单元与集成测试、前端生产构建和前端单元测试。对应命令分别为 mvn -f backend/pom.xml test、npm --prefix frontend run build 和 npm --prefix frontend run test:unit -- --run。